Métodos Numéricos y Optimización
Lecture 1: Nested arithmetic, análisis de convergencia & métodos de búsqueda de raíces

Martin Palazzo

Universidad de San Andrés
Ingeniería en Inteligencia Artificial

palazzom@udesa.edu.ar

February 28, 2024

Overview

1. Nested arithmetic in Polynomials
2. Algoritmos: convergencia
3. Root finding problems
4. Método de la bisección
5. Lectura sugerida de esta clase

Nested arithmetic in Polynomials

Aritmetica punto flotante en polinomios

Como hemos visto en ejemplos anteriores la representacion de punto flotante genera un error > 0 que puede ser arrastrado en mayor o menor medida cuando realizamos aritmetica. Concretamente estudiaremos el caso de aritmetica en polinomios.

Ejemplo

Supongamos que queremos evaluar en $x = 4.71$ el polinomio

$$f(x) = x^3 - 6.1x^2 + 3.2x + 1.5$$

utilizando operaciones aritmeticas de 3 digitos.

Aritmetica punto flotante en polinomios

Ejemplo

Supongamos que queremos evaluar en $x = 4.71$ el polinomio

$$f(x) = x^3 - 6.1x^2 + 3.2x + 1.5$$

utilizando operaciones aritmeticas de 3 digitos.

Calculos preliminares (utilizando $k = 3$ digitos)

$(x = 4.71)$	x	x^2	x^3	$6.1x$	$3.2x$
Exacto	4.71	22.1841	104.487111	135.32301	15.072
Chopping	4.71	22.1	104	134	15.0
Rounding	4.71	22.2	105	135	15.1

Aritmetica punto flotante en polinomios

Calculos preliminares (utilizando $k = 3$ digitos)

$(x = 4.71)$	x	x^2	x^3	$6.1x$	$3.2x$
Exacto	4.71	22.1841	104.487111	135.32301	15.072
Chopping	4.71	22.1	104	134	15.0
Rounding	4.71	22.2	105	135	15.1

Resultados con distintas estrategias

$$
\begin{aligned}
f(4.71)_{\textbf{Exact}} &= ((104.487111 - 135.32301) + 15.072) + 1.5 = -14.26389 \\
f(4.71)_{\textbf{Chop}} &= ((104 - 134) + 15) + 1.5 = -13.5 \\
f(4.71)_{\textbf{Round}} &= ((105 - 135) + 15.1) + 1.5 = -13.4
\end{aligned}
$$

Aritmetica punto flotante en polinomios

Resultados con distintas estrategias

$$\begin{aligned} f(4.71)_{\textbf{Exact}} &= -14.26389 \\ f(4.71)_{\textbf{Chop}} &= -13.5 \\ f(4.71)_{\textbf{Round}} &= -13.4 \end{aligned}$$

Analisis de error

$$\text{Error relativo chopping} = \frac{|f(4.71)_{\textbf{Exact}} - f(4.71)_{\textbf{Chop}}|}{f(4.71)_{\textbf{Exact}}} \approx 0.05$$

$$\text{Error relativo rounding} = \frac{|f(4.71)_{\textbf{Exact}} - f(4.71)_{\textbf{Round}}|}{f(4.71)_{\textbf{Exact}}} \approx 0.06$$

Observar que para **aproximar** el polinomio con aritmetica de $k = 3$ digitos el error relativo en redondeo no llega a 2 digitos significativos!

Aritmetica punto flotante en polinomios

Nested arithmetic

Podemos mejorar estos resultados para el caso de un polinomio? La respuesta es afirmativa y se basa en re-ordenar la expresion polinomica de manera *anidada* reescribiendo el polinomio de la siguiente manera

$$f(x) = x^3 - 6.1x^2 + 3.2x + 1.5 = ((x - 6.1)x + 3.2)x + 1.5$$

Aritmetica punto flotante en polinomios

Nested arithmetic

Entonces si ahora la función inicial queda definida como

$$f(x) = ((x - 6.1)x + 3.2)x + 1.5$$

entonces

$$\begin{aligned} f_{\textbf{chop}}(4.71) &= ((4.71 - 6.1)4.71 + 3.2)4.71 + 1.5 = \quad -14.2 \\ f_{\textbf{chop}}(4.71) &= ((4.71 - 6.1)4.71 + 3.2)4.71 + 1.5 = \quad -14.3 \end{aligned}$$

Análisis de error para *Nested arithmetic*

$$\text{Error relativo chopping} = \frac{|f(4.71)_{\textbf{Exact}} - f(4.71)_{\textbf{Chop}}|}{f(4.71)_{\textbf{Exact}}} \approx 0.0045$$

$$\text{Error relativo rounding} = \frac{|f(4.71)_{\textbf{Exact}} - f(4.71)_{\textbf{Round}}|}{f(4.71)_{\textbf{Exact}}} \approx 0.0025$$

Aritmética punto flotante en polinomios

Cual fue la razón de una reducción de mas del 95% en el error absoluto por redondeo?

Conclusión

- En la propuesta inicial $f(x) = x^3 - 6.1x^2 + 3.2x + 1.5$ hay 5 multiplicaciones y 3 sumas ($x^3 = x \cdot x \cdot x$).
- En la propuesta "*anidada*" $f(x) = ((x - 6.1)x + 3.2)x + 1.5$ tenemos 2 multiplicaciones y 3 sumas.

Algoritmos: convergencia

Algoritmos

Durante la materia de Metodos Numericos y Optimizacion estudiaremos distintos **metodos iterativos** para poder aproximar soluciones matematicas con la maquina/computadora.

Algoritmos

Es un procedimiento que describe una secuencia finita de n pasos o iteraciones a ser ejecutados en un orden especifico [1]. En muchos casos los algoritmos repiten secuencias de pasos en forma de "*loop*". El enfoque de *looping* utilizado puede categorizarse entre

- *counter-controlled* donde la cantidad de ciclos de loop es un numero finito.
- *condition-controlled* donde el algoritmo realizara ciclos siempre que se encuentre activa una condición *boolean* (True-False) especifica.

Algoritmos

Si quisieramos sumar $x_1 + x_2 + \ldots x_n = \sum_{i=1}^{i=N} x_i$ el algoritmo podemos estructurarlo de la siguiente forma

INPUT $N, x_1, x_2, \ldots, x_n$.

OUTPUT $SUM = \sum_{i=1}^{N} x_i$.

Step 1 Set $SUM = 0$. (*Initialize accumulator.*)

Step 2 For $i = 1, 2, \ldots, N$ do
set $SUM = SUM + x_i$. (*Add the next term.*)

Step 3 OUTPUT (SUM);
STOP.

Figure: Algoritmo que suma N numeros.

Convergencia algorítmica

Para resolver un problema con la maquina podemos utilizar múltiples métodos numéricos estructurados como algoritmos. Un aspecto para evaluar el rendimiento de estos algoritmos es analizar cuan rápido convergen a una solución dada.

Rendimiento de métodos numéricos

- cuantas n iteraciones hay que realizar para llegar a una solucion que este acotada a un error determinado?
- Como evoluciona el error en función de las n iteraciones que realicemos?

Convergencia algorítmica

Supongamos que tenemos la secuencia $\{x_n\}_{n=1}^{\infty}$ cuyo limite

$$\lim_{n\to\infty} x_n = 0$$

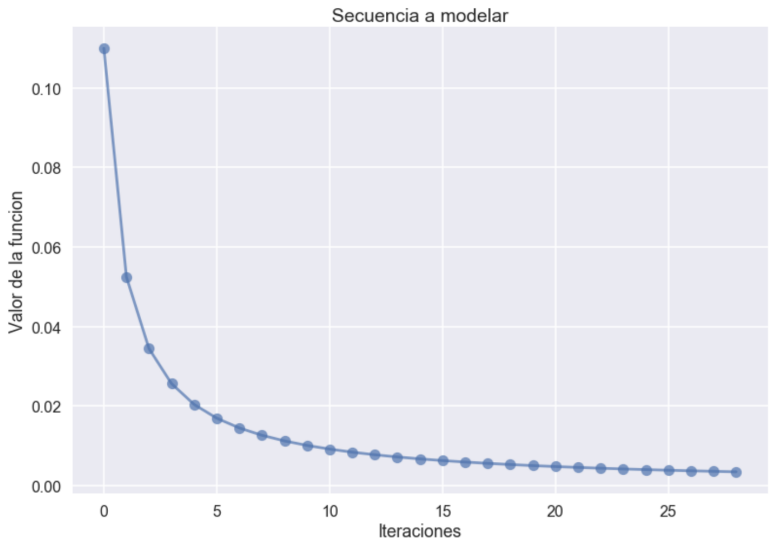


Figure: Secuencia que converge a 0

Dado que con una maquina nunca podremos asegurar $\lim_{n\to\infty}$, cuantos pasos debemos dar para llegar a una aproximación significativa del limite teórico?

Convergencia algorítmica

Definición

Sean $\{x_n\}_{n=1}^{\infty}$ e $\{y_n\}_{n=1}^{\infty}$ secuencias que satisfacen

$$\lim_{n\to\infty} x_n = \alpha, \ \lim_{n\to\infty} y_n = 0$$

donde $x \in \mathbb{R}$. Se determina que $\{x_n\}$ converge a α con **tasa de convergencia** $O(y_n)$ si existe $M \in \mathbb{R}^+$ tal que

$$|x_n - \alpha| \quad \leq M|y_n|$$

y un n_0 que asegure $\forall n \geq n_0$.

Función de comparación

Vamos a utilizar $y_n = \frac{1}{n^p}$ con $p > 0$ y para expresar como $x_n = x + O(\frac{1}{n^p})$ la convergencia del metodo.

Convergencia algorítmica

Definición

La secuencia $\{x_n\}$ converge a su limite α con un **orden** $j \geq 1$ si

$$|\alpha - x_{n+1}| \leq c|\alpha - x_n|^j$$

para algún $c > 0$ (llamada constante de error asintótica). Por ejemplo, si $j = 1$ la secuencia **converge linealmente** a α.

Conclusión

Una secuencia de alto orden de convergencia va a converger mas rápido que una secuencia de orden bajo.

Convergencia algorítmica

Supongamos la secuencia $x_n = \frac{n+1}{n+2}$. Sabemos que $\lim_{n\to\infty} x_n = 1$.

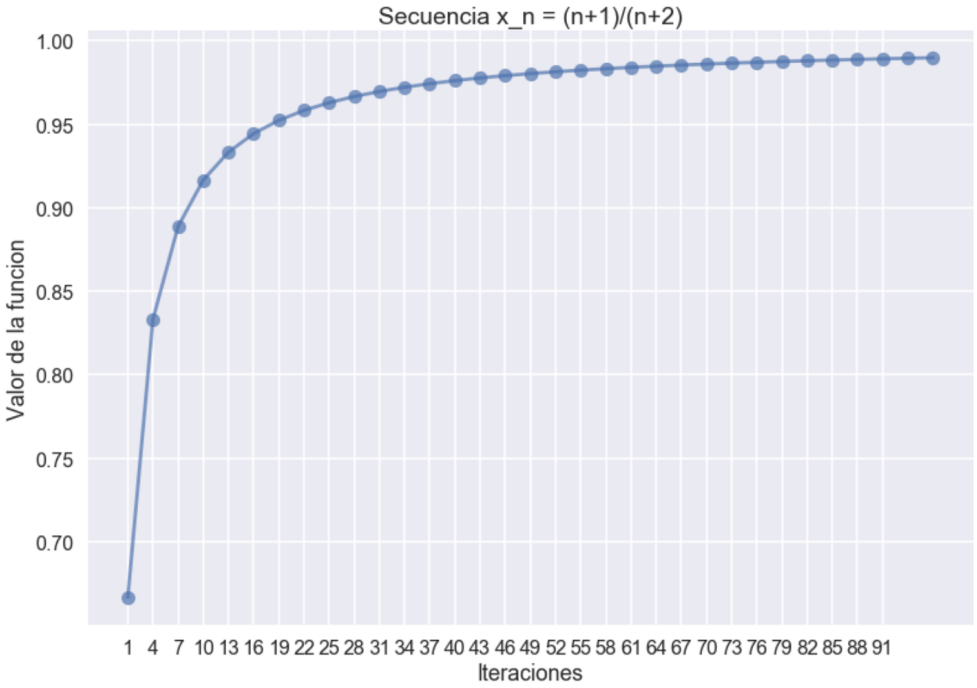


Figure: Secuencia $\frac{n+1}{n+2}$ que converge a 1

Podemos estimar su tasa de convergencia para un p dado.

Convergencia algorítmica

Para nuestro ejemplo $x_n = \frac{n+1}{n+2}$ si $y_n = \frac{1}{n}$ con $p = 1$ observamos empíricamente que $|x_n - 1| \leq \frac{1}{n^p}$ entonces $x_n = 1 + O(\frac{1}{n})$

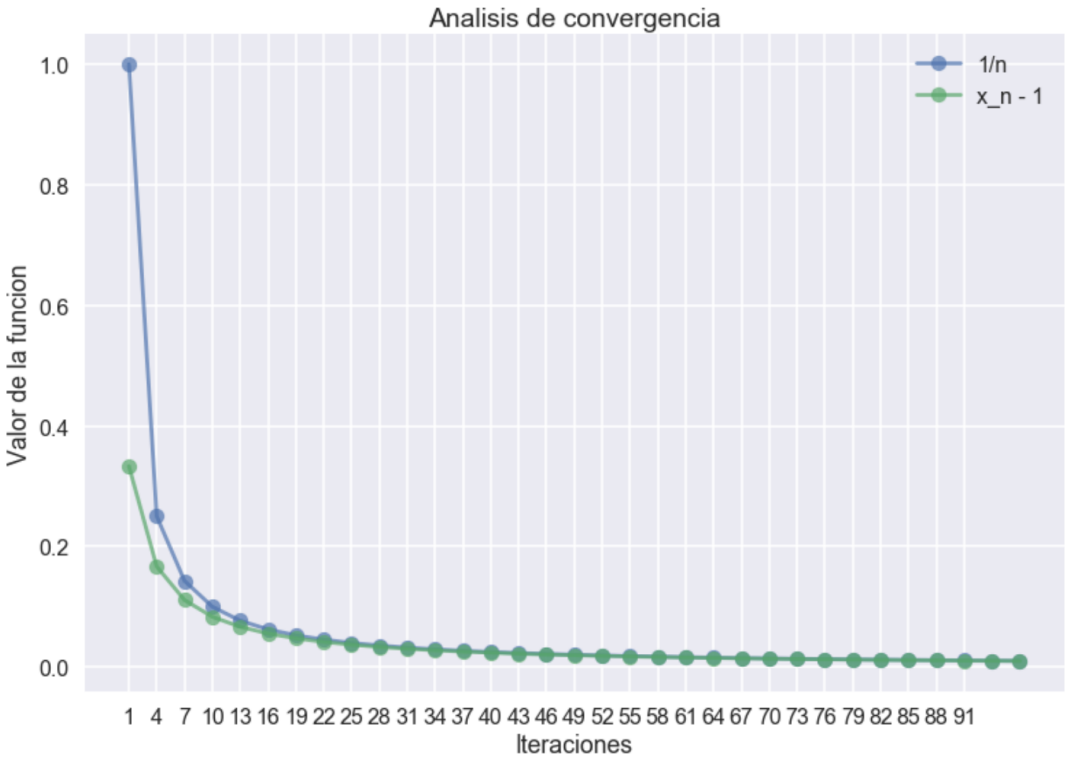


Figure: Secuencia $\frac{n+1}{n+2}$ y su tasa de convergencia

Convergencia algorítmica

Supongamos las secuencias $x_\delta = \frac{n+1}{n^2}$ y $x_\beta = \frac{n+3}{n^3}$. Sabemos que

- $\lim_{n\to\infty}\{x_\delta\} = 0$
- $\lim_{n\to\infty}\{x_\beta\} = 0$

Cuales son las tasas de convergencia de ambas secuencias? Cual converge mas rápidamente al limite?

Convergencia algorítmica

Visualmente podemos observar lo siguiente

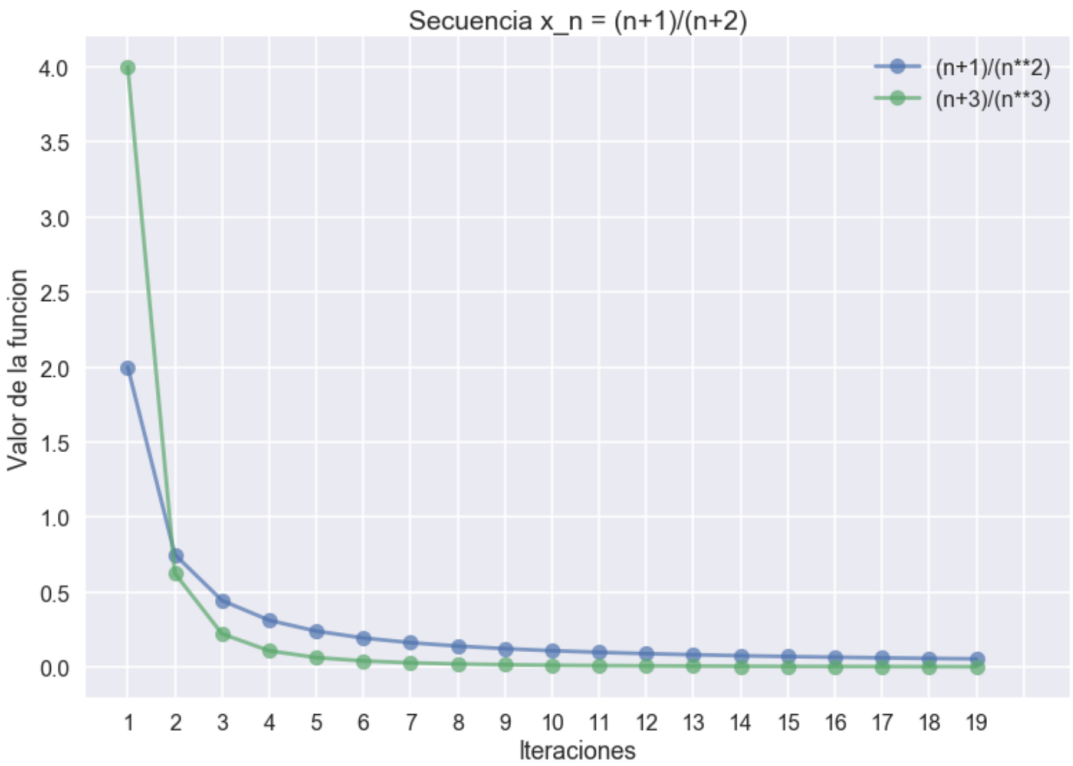


Figure: Secuencias $\frac{n+1}{n^2}$ y $\frac{n+3}{n^3}$

Convergencia algorítmica

Analíticamente buscaremos acotar la tasa de convergencia de ambas secuencias a una expresión similar a $\frac{1}{n^p}$ entonces

Convergencia x_δ

$$|x_\delta - 0| = \frac{n+1}{n^2} \leq \frac{n+n}{n^2} = 2 \cdot \frac{1}{n} \rightarrow x_\delta = 0 + O(\frac{1}{n})$$

Convergencia x_β

$$|x_\beta - 0| = \frac{n+3}{n^3} \leq \frac{n+3n}{n^3} = 4 \cdot \frac{1}{n^2} \rightarrow x_\delta = 0 + O(\frac{1}{n^2})$$

Root finding problems

Root finding problems

Un problema recurrente que vamos a querer resolver numéricamente (es decir, con la maquina y no analíticamente) es $g(x) = h(x)$.

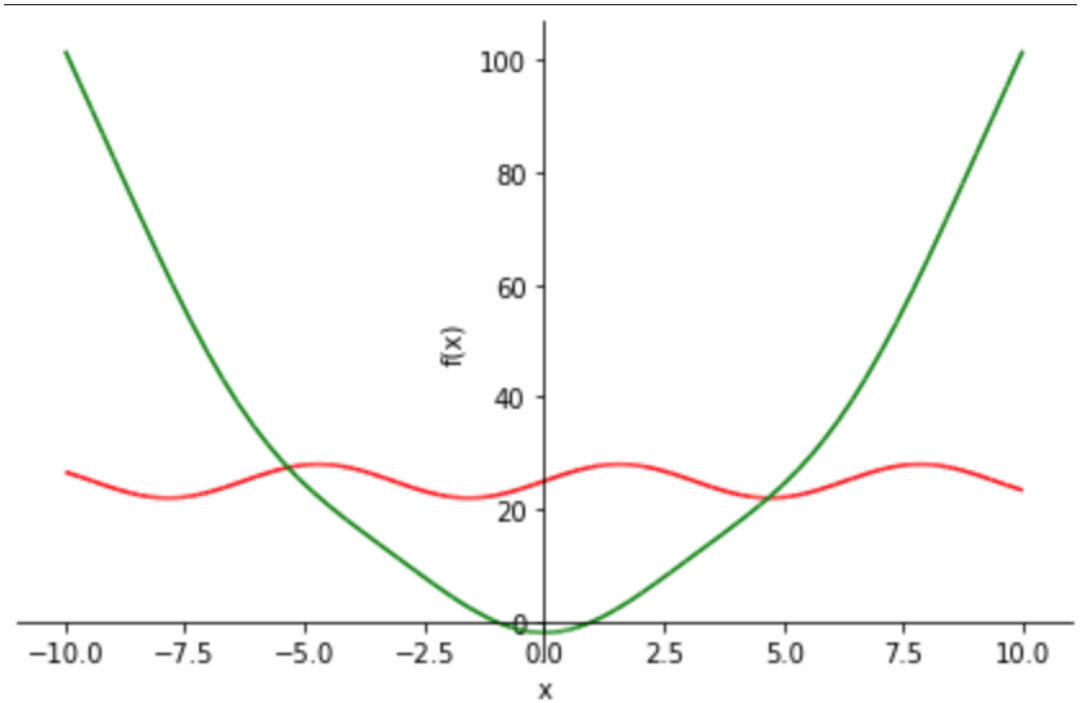


Figure: $g(x) = 3sin(x) + 25$ y $h(x) = x^2 - 2cos(x)$.

Root finding problems

Este problema fácilmente puede reescribirse como

$$f(x) = g(x) - h(x)$$

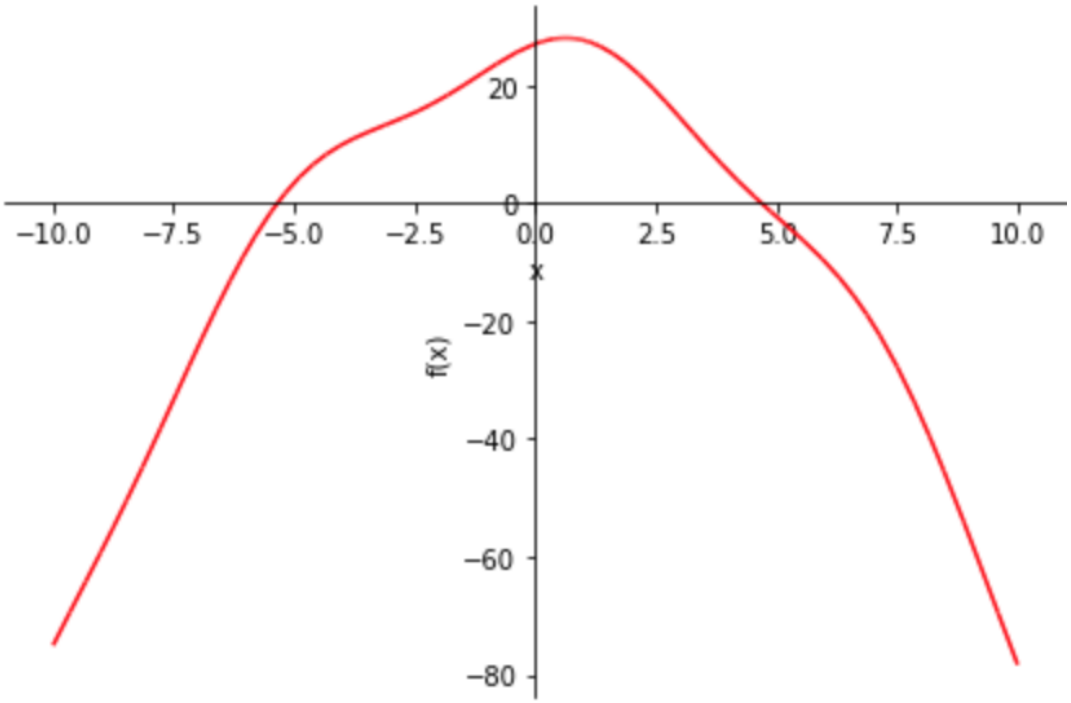


Figure: $f(x) = 3sin(x) + 25 - (x^2 - 2cos(x))$.

donde $f(x = r) = 0$

Root finding problems

Vamos a querer encontrar la raiz de

$$f(x = r) = 0$$

para ello estudiaremos tres métodos de interés (entre otros)

- Método de la bisección
- Método de punto fijo
- Método de Newton-Raphson

Método de la bisección

Método de la bisección

Supongamos que $f(x)$ es continua en el intervalo $[a, b]$ el teorema del valor intermedio asegura que

Teorema del valor intermedio

Si $f \in C[a, b]$ y $f(a) < K < f(b)$ entonces existe $r \in (a, b)$ que asegura $f(r) = K$. Si $K = 0$ y

$$f(a) \cdot f(b) < 0$$

entonces $\exists\ r$ con $a < r < b$ asegurando $f(r) = 0$ y asumimos que dicha raiz es unica (por mas que no sea cierto).

Método de la bisección

Dada la funcion continua $f(x)$, para encontrar una solucion a $f(x) = 0$ en el intervalo $[a_1, b_1]$ donde $f(a) \cdot f(b) < 0$ el algoritmo utiliza la tecnica de la biseccion y comienza asumiendo que la raiz se encuentra a la mitad entre a y b

$$r_1 = \frac{a_1 + b_1}{2}$$

entonces

- si $f(r_1) = 0$ el metodo iterativo termina
- si $f(r_1) \neq 0$ y $f(r_1) \cdot f(a_1) > 0$ then $r \in (r_1, b_1) \rightarrow a_1 = r_1,\ b_2 = b_1$
- si $f(r_1) \neq 0$ y $f(r_1) \cdot f(b_1) > 0$ then $r \in (r_1, a_1) \rightarrow b_1 = r_1,\ a_2 = a_1$

se vuelve a aplicar el metodo en el intervalo $[a_2, b_2]$. Repetir n veces

Método de la bisección

Podemos definir una tolerancia $\epsilon > 0$ para generar iterativamente multiples raices $r_1, \ldots, r_n$ hasta que se cumpla alguna de las condiciones

Condiciones de terminacion

- $|r_n - r_{n-1}| < \epsilon$
- $|f(r_n)| < \epsilon$
- $\frac{|r_n - r_{n-1}|}{|r_n|} < \epsilon$

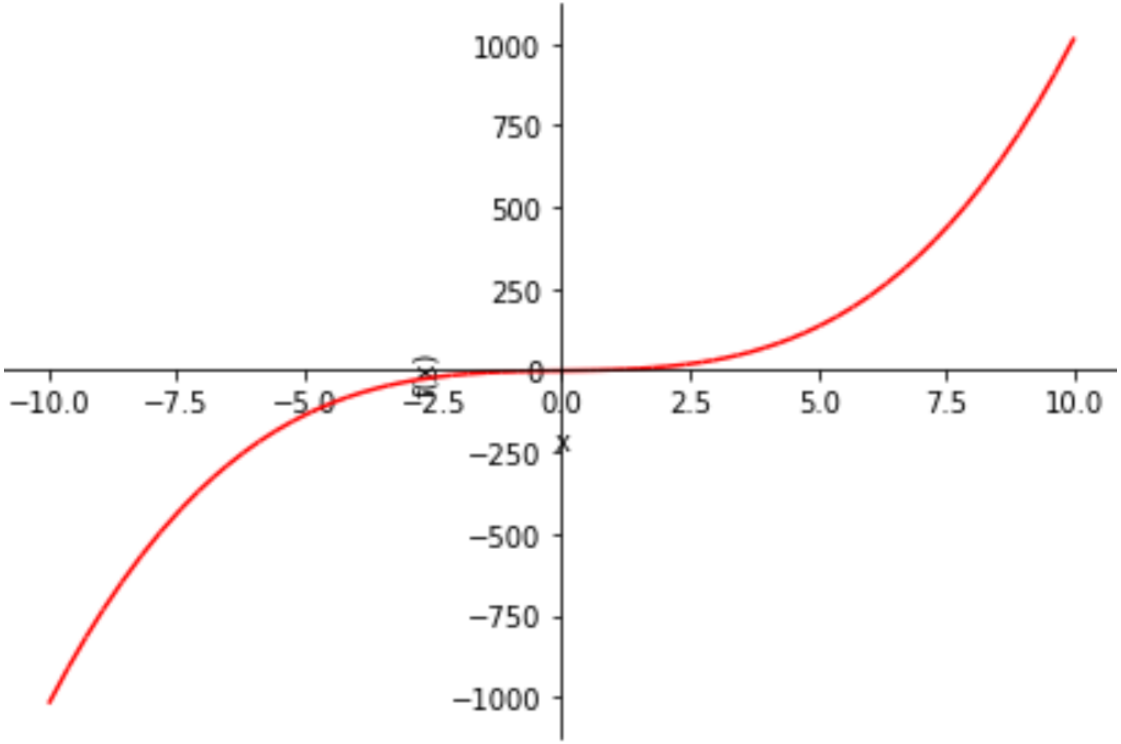
También podemos terminar el método una vez que se cumplan N iteraciones.

Método de la bisección

Algorithm Calcular $f(x) = 0$ con método de la bisección

```
Require: f(x) ∧ ϵ > 0 ∧ n > 0 ∧ a ∧ b
  if f(a) · f(b) > 0 then
      print("Wrong a,b range")
  else
      while i ≤ N do
          r := (a + b)/2
          if f(a) · f(r) < 0 then
              b = r
          else
              a = r
          end if
          if f(r_i) < ϵ ∨  then
              i = N
          end if
          i = i + 1
      end while
      return r
  end if
```

Método de la bisección: ejemplo



Ejemplo

Dada la siguiente funcion encontrar su raiz con el metodo de la biseccion

$$f(x) = x^3 + 2x - 1$$

Método de la bisección

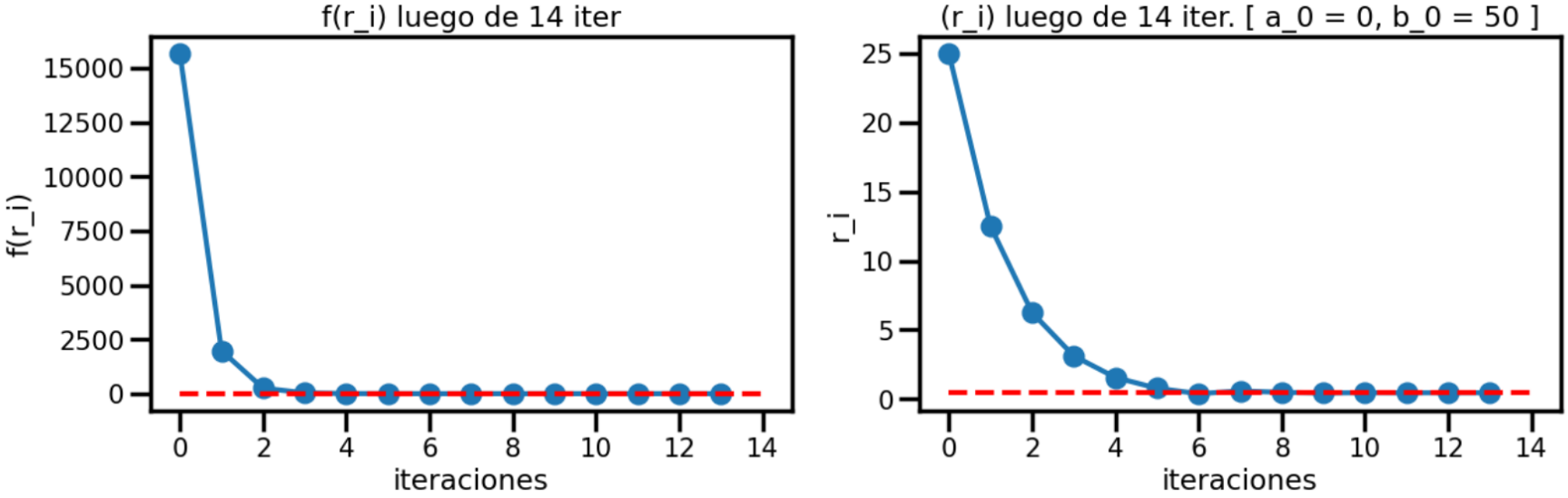
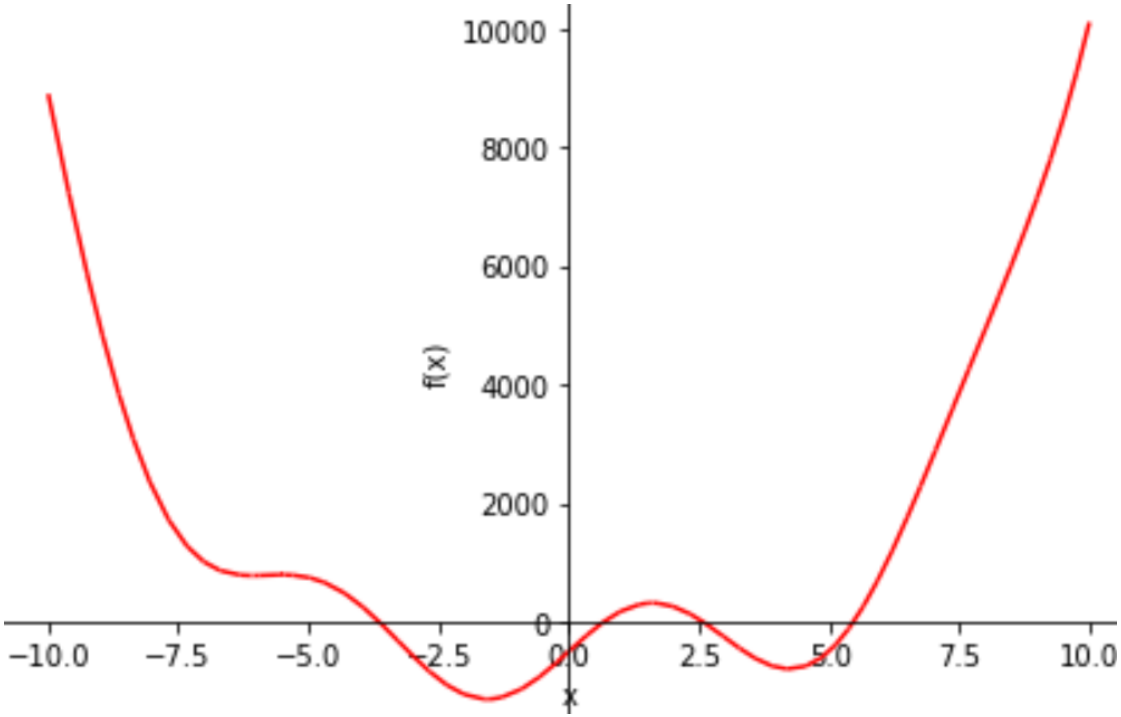


Figure: $f(x) = x^3 + 2x - 1 = 0.$

Metodo de la biseccion: ejemplo



Ejemplo

Dada la siguiente funcion encontrar su raiz con el metodo de la biseccion

$$f(x) = x^4 + x^3 + 4x - 500 + 800sin(x)$$

Metodo de la biseccion: curva convergencia

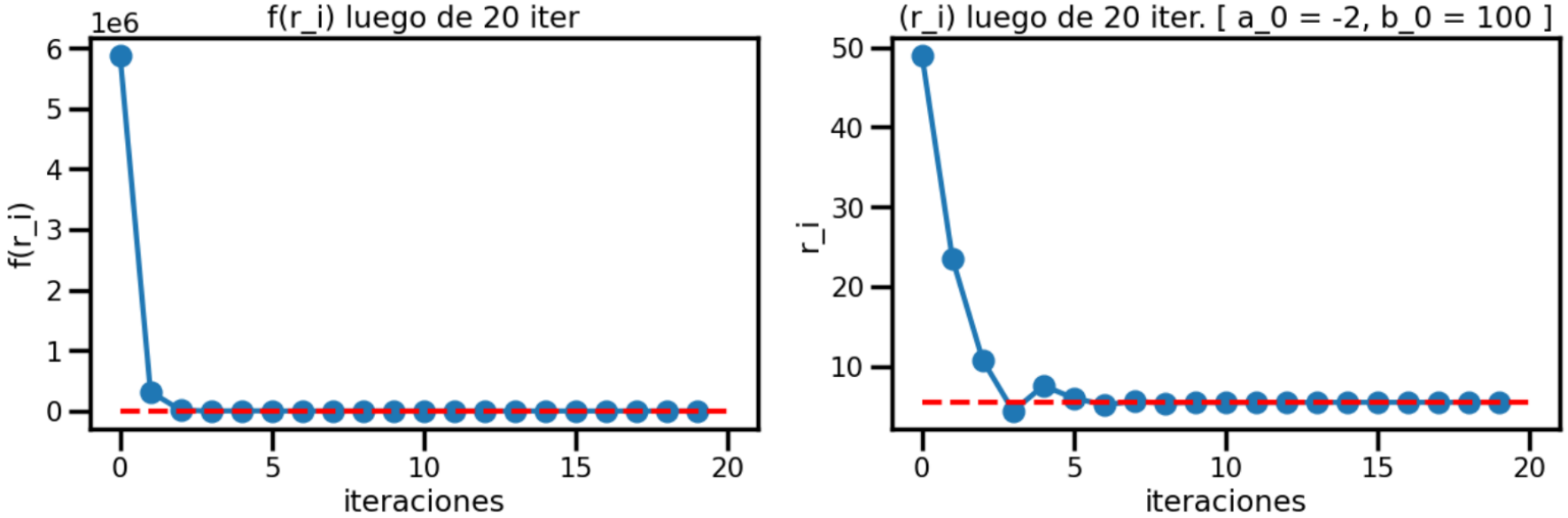


Figure: Búsqueda de raíz de $f(x) = x^4 + x^3 + 4x - 500 + 800sin(x)$.

Si tomamos $a = -2$ y $b = 100$ se da que $f(a) \cdot f(b) < 0$

Método de la bisección: tabla convergencia

	Raiz r_i	f(r_i)
0	49.000000	5.881383e+06
1	23.500000	3.167535e+05
2	10.750000	1.336399e+04
3	4.375000	-7.872940e+02
4	7.562500	3.999872e+03
5	5.968750	7.583041e+02
6	5.171875	-3.425271e+02
7	5.570312	1.346717e+02
8	5.371094	-1.239509e+02
9	5.470703	5.367963e-01
10	5.420898	-6.293801e+01
11	5.445801	-3.150535e+01
12	5.458252	-1.556008e+01
13	5.464478	-7.530540e+00
14	5.467590	-3.501591e+00
15	5.469147	-1.483576e+00
16	5.469925	-4.736847e-01
17	5.470314	3.148214e-02
18	5.470119	-2.211197e-01
19	5.470217	-9.482337e-02

Búsqueda de raíz de $f(x) = x^4 + x^3 + 4x - 500 + 800sin(x)$ con el método de la bisección.

Método de la bisección: cota de error aproximación

Teorema: cota al error de aproximacion

Si $f \in C[a,b]$ y $f(a) \cdot f(b) < 0$ el método de la bisección generara una secuencia de soluciones $p_n{}_{n=1}^{\infty}$ en cada iteración que aproxima a la raiz p de f con

$$p_n - p \leq \frac{b-a}{2^n}$$

con $n \geq 1$

Teorema cota al error de aproximación: demostracion

Sabemos que

$$b_n - a_n = \frac{1}{2^{n-1}}(b-a)$$

y que $p \in (a_n, b_n)$. Sabemos que $p_n = \frac{1}{2}(a_n + b_n)$ entonces

$$|p_n - p| \leq \frac{1}{2}(b_n - a_n) = \frac{b-a}{2^n}$$

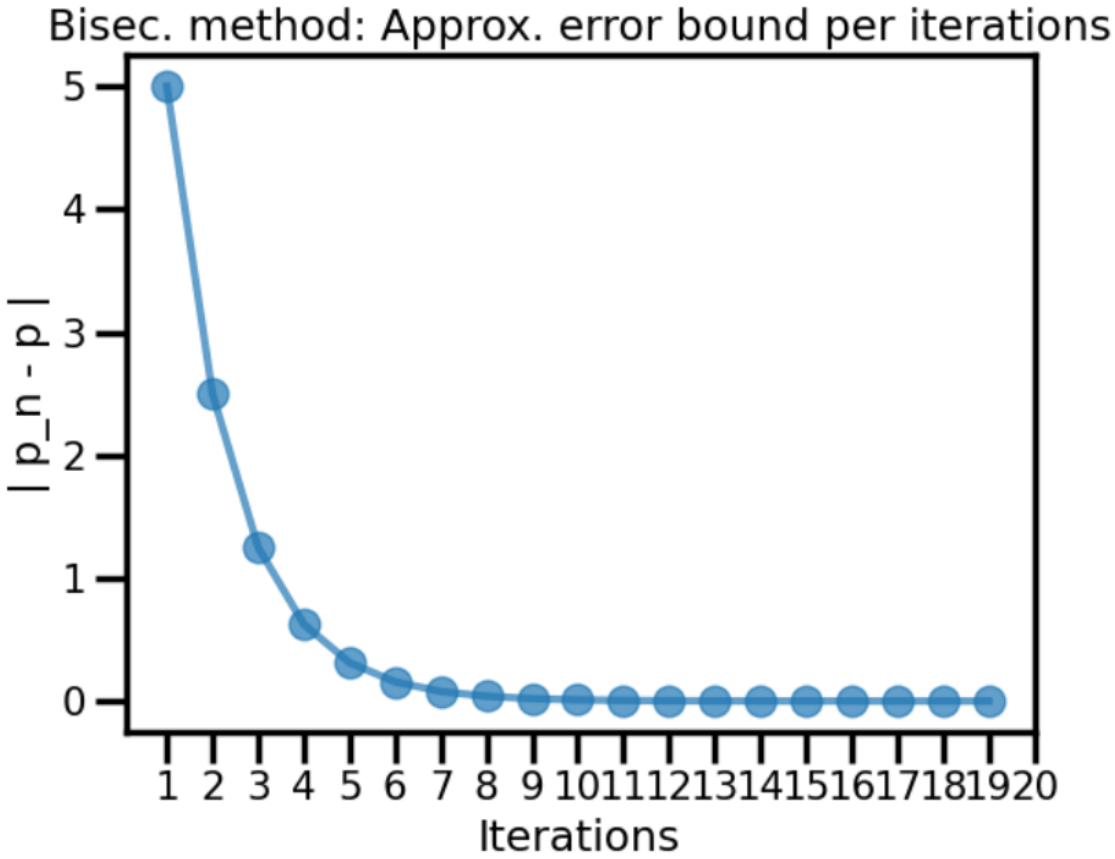
Por lo tanto si

$$|p_n - p| \leq \frac{b-a}{2^n}$$

entonces la secuencia $p_n{}_{n=1}^{\infty}$ converge a p a una tasa de convergencia $O(\frac{1}{2^n})$ siendo

$$p_n = p + O(\frac{1}{2^n})$$

Método de la bisección: cota al error de aproximación



Cota al error de aproximación a través de n iteraciones

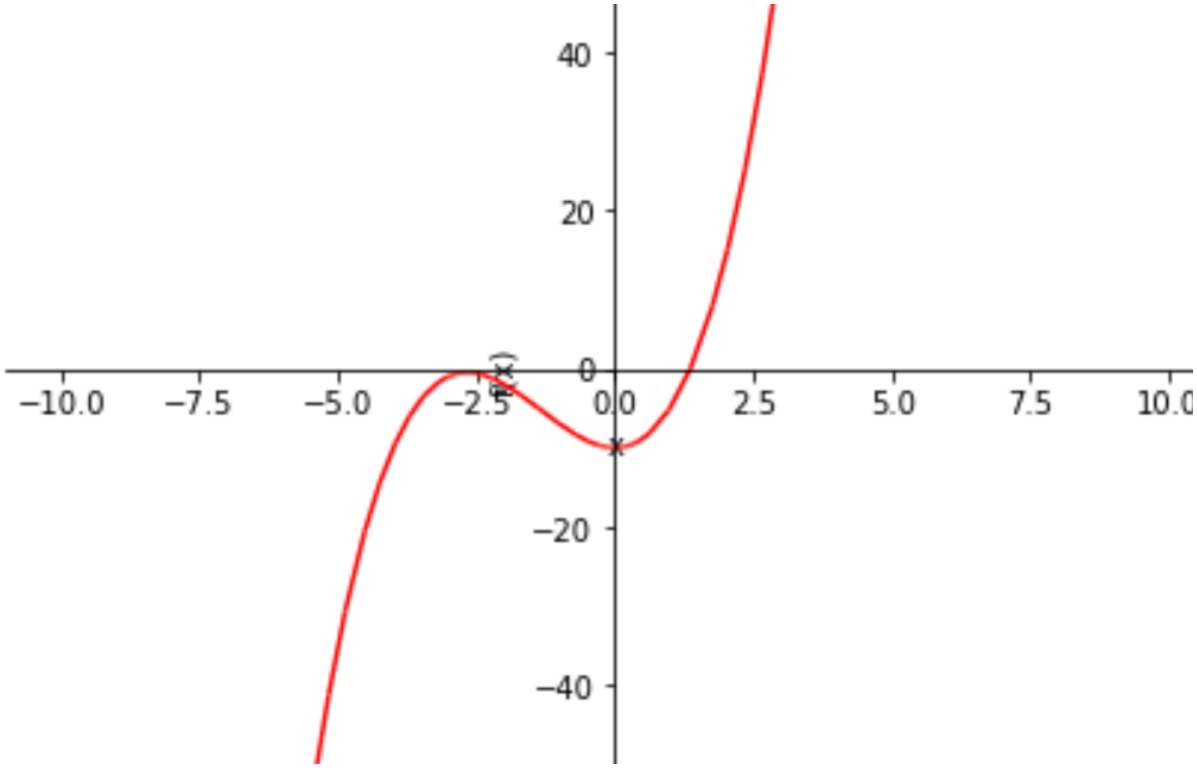
$$|p_n - p| \leq \frac{b-a}{2^n}$$

Calcular iteraciones necesarias

Ejercicio

Supongamos que tenemos $f(x) = x^3 + 4x^2 - 10$ y queremos calcular la cantidad de iteraciones n necesarias para encontrar $f(x) = 0$ con una precision de 10^{-3} sabiendo que $a_1 = 1$ y $b_1 = 2$

Calcular iteraciones necesarias



$$f(x) = x^3 + 4x^2 - 10$$

Calcular las iteraciones necesarias

Resolución

Por el teorema de cota al error de aproximación sabemos que

$$|p_n - p| \leq \frac{b-a}{2^N} = \frac{1}{2^N} \leq 10^{-3}$$

si aplicamos logaritmos (base-10)

$$-N \log_{10}(2) \leq -3$$

obtenemos

$$N > \frac{3}{\log_{10}(2)} \approx 9.96 \rightarrow N = 10$$

Método de la bisección: algunas conclusiones

conclusiones

- El metodo de la biseccion siempre **converge** a una solucion.
- Si bien pueden existir metodos que convergen mas rapido, el metodo sirve como *baseline* para comparar.
- La cota calculada para la precisión de aproximación es conservadora y muchas veces puede estar sobre-dimensionada.

Lectura sugerida de esta clase

Lecturas sugeridas

Lectura sugeridas

- Convergencia: Capítulo 1.3 libro Burden Faires
- Método de la bisección: Capítulo 2.1 libro Burden Faires

Burden, R. L., Faires, J. D., & Burden, A. M. (2015). Numerical analysis. Cengage learning.